LAB 8 · LECTURE 8

Lecture 8 Lab: Ask Your Files, a RAG App

Vector Databases Course | Estimated time: 90 minutes | Platform: Google Colab (free) | Everything in this lab is free

Goal

You will build a chat-with-your-PDF application using only free tools: Chroma for retrieval, the open-weight model Qwen2.5-1.5B-Instruct (Apache-2.0) for generation, and Gradio for the interface. No API key and no billing account are needed.

- Ingest a PDF into a persistent Chroma collection with page metadata.
- Choose a distance threshold from real measurements, so the app can refuse.
- Load a free open model and write a grounded prompt with numbered citations.
- Answer follow-up questions by rewriting them as standalone questions.
- Validate citations, time each stage, and run a small smoke test.
- Launch a web interface with Gradio.

Setup

Create a new Colab notebook. If you can, choose Runtime, Change runtime type, and select a free GPU; the lab also runs on CPU, only slower. Run each block in its own cell, in order.

Cell 1: install and import

```
!pip install -q chromadb sentence-transformers pypdf requests gradio
!pip install -q transformers accelerate

import re, json, time
import numpy as np, pandas as pd
import requests, torch
import chromadb
from chromadb.utils import embedding_functions as ef
from pypdf import PdfReader
print("GPU available:", torch.cuda.is_available())
```

Part 1: Ingest the PDF

This reuses the cleaning and recursive chunking from Lab 6. If you kept `chunks.jsonl`, you can load it instead.

Cell 2: download, clean and chunk

```
url = "https://arxiv.org/pdf/1706.03762"
r = requests.get(url, headers={"User-Agent": "Mozilla/5.0"}, timeout=60)
open("paper.pdf", "wb").write(r.content)
pages = [p.extract_text() or "" for p in PdfReader("paper.pdf").pages]
print(len(pages), "pages")

def clean(text):
    text = re.sub(r"-\n(?=[a-z])", "", text)       # rejoin hyphenated line breaks
    text = re.sub(r"(?m)^\s*\d+\s*$", "", text)     # drop lines with only a number
    text = re.sub(r"[ \t]+", " ", text)
    return text.strip()

def recursive_chunks(text, size=800, seps=("\n\n", "\n", ". ", " ")):
    if len(text) <= size:
        return [text]
    for k, sep in enumerate(seps):
        if sep in text:
            parts = [p + sep for p in text.split(sep)]
            parts[-1] = parts[-1][:-len(sep)]
            rest = seps[k + 1:]
            break
    else:
        return [text[i:i + size] for i in range(0, len(text), size)]
    chunks, cur = [], ""
    for part in parts:
        if len(part) > size:
            if cur:
                chunks.append(cur); cur = ""
            chunks.extend(recursive_chunks(part, size, rest))
        elif len(cur) + len(part) <= size:
            cur += part
        else:
            chunks.append(cur); cur = part
    if cur:
        chunks.append(cur)
    return [c.strip() for c in chunks if c.strip()]
```

Cell 3: store the chunks in Chroma

```
minilm = ef.SentenceTransformerEmbeddingFunction(model_name="all-MiniLM-L6-v2")
client = chromadb.PersistentClient(path="rag_db")
col = client.get_or_create_collection("paper", embedding_function=minilm,
                                      configuration={"hnsw": {"space": "cosine"}})

records = []
for pno, ptext in enumerate(pages, start=1):
    for j, c in enumerate(recursive_chunks(clean(ptext), size=800)):
        records.append({"id": f"paper:p{pno}:{j}", "text": c, "page": pno})

col.upsert(ids=[r["id"] for r in records],
           documents=[r["text"] for r in records],
           metadatas=[{"source": "paper.pdf", "page": r["page"]} for r in records])
print(col.count(), "chunks stored")
```

Part 2: Retrieval and the refusal threshold

Cell 4: retrieve, and measure distances for both kinds of question

```
def retrieve(question, k=4):
    res = col.query(query_texts=[question], n_results=k)
    return [{"text": d, "source": m["source"], "page": m["page"], "distance": dist}
            for d, m, dist in zip(res["documents"][0], res["metadatas"][0],
                                  res["distances"][0])]

answerable = ["Which optimizer was used for training?",
              "What hardware was the model trained on?",
              "How many identical layers does the encoder have?",
              "How is attention computed from queries, keys and values?"]
unanswerable = ["What is the capital of France?",
                "Who won the 2018 football World Cup?",
                "How do I bake sourdough bread?"]

rows = []
for kind, qs in [("answerable", answerable), ("unanswerable", unanswerable)]:
    for q in qs:
        rows.append({"kind": kind, "question": q,
                     "best_distance": round(retrieve(q, 1)[0]["distance"], 3)})
pd.DataFrame(rows)
```

YOUR TURN Look at the best distances. Pick a threshold that separates the two groups, and write it down with your reasoning. Is there a clean gap? What would you do if the groups overlapped?

Cell 5: set the threshold

```
MAX_DISTANCE = 0.6        # replace with the value you chose above
REFUSAL = "I don't know based on the documents."
```

Part 3: A free generator

Qwen2.5-1.5B-Instruct is released under the Apache-2.0 license. The download is a few gigabytes and takes a few minutes the first time. If memory runs out, use Qwen/Qwen2.5-0.5B-Instruct instead.

Cell 6: load the model and a generate function

```
from transformers import AutoTokenizer, AutoModelForCausalLM

name = "Qwen/Qwen2.5-1.5B-Instruct"
tok = AutoTokenizer.from_pretrained(name)
llm = AutoModelForCausalLM.from_pretrained(name, torch_dtype="auto", device_map="auto")

def generate(messages, max_new_tokens=200):
    text = tok.apply_chat_template(messages, tokenize=False, add_generation_prompt=True)
    inputs = tok(text, return_tensors="pt").to(llm.device)
    with torch.no_grad():
        out = llm.generate(**inputs, max_new_tokens=max_new_tokens, do_sample=False)
    return tok.decode(out[0][inputs["input_ids"].shape[1]:],
                      skip_special_tokens=True).strip()

print(generate([{"role": "user", "content": "Say hello in one sentence."}]))
```

Part 4: The RAG function

Cell 7: prompt, ask, and a display helper

```
SYSTEM = ("Answer using only the numbered context passages. Cite passages like [1]. "
          f"If the context does not contain the answer, reply exactly: {REFUSAL}")

def build_messages(question, hits):
    ctx = "\n\n".join(f"[{i}] ({h['source']}, page {h['page']})\n{h['text']}"
                      for i, h in enumerate(hits, start=1))
    user = f"Context:\n{ctx}\n\nQuestion: {question}"
    return [{"role": "system", "content": SYSTEM}, {"role": "user", "content": user}]

def ask(question, k=4, max_distance=None):
    limit = MAX_DISTANCE if max_distance is None else max_distance
    t0 = time.perf_counter()
    hits = [h for h in retrieve(question, k) if h["distance"] <= limit]
    t1 = time.perf_counter()
    if not hits:
        return {"answer": REFUSAL, "hits": [], "t_retrieve": t1 - t0, "t_generate": 0.0}
    answer = generate(build_messages(question, hits))
    return {"answer": answer, "hits": hits, "t_retrieve": t1 - t0,
            "t_generate": time.perf_counter() - t1}

def show_answer(question):
    r = ask(question)
    print("Q:", question)
    print("A:", r["answer"])
    for i, h in enumerate(r["hits"], start=1):
        print(f"   [{i}] {h['source']} p{h['page']}  distance {h['distance']:.2f}")
    ms = r["t_retrieve"] * 1000
    print(f"   retrieve {ms:.0f} ms | generate {r['t_generate']:.1f} s")

for q in answerable[:2] + unanswerable[:1]:
    show_answer(q)
```

YOUR TURN For each answer, is it correct and is every claim supported by a retrieved chunk? Record the retrieval and generation times. Which stage dominates, and what would you change to speed it up?

Part 5: Follow-ups and citation checks

Cell 8: standalone questions and citation validation

```
def condense(history, followup):
    if not history:
        return followup
    convo = "\n".join(f"User: {q}\nAssistant: {a}" for q, a in history[-2:])
    user = f"{convo}\nUser: {followup}\nStandalone question:"
    rule = "Rewrite the last question as a standalone question. Output only it."
    msg = [{"role": "system", "content": rule},
           {"role": "user", "content": user}]
    return generate(msg, max_new_tokens=60)

def citations_ok(answer, n_hits):
    cited = [int(x) for x in re.findall(r"\[(\d+)\]", answer)]
    return len(cited) > 0 and all(1 <= c <= n_hits for c in cited)

history = []
q1 = "Which optimizer was used for training?"
a1 = ask(q1); history.append((q1, a1["answer"]))
standalone = condense(history, "What learning rate schedule did it use?")
print("rewritten:", standalone)
a2 = ask(standalone)
print(a2["answer"], "| citations ok:", citations_ok(a2["answer"], len(a2["hits"])))
```

YOUR TURN Compare retrieval for the raw follow-up question with retrieval for the rewritten one (call retrieve on both). What changes in the top chunk and its distance?

Part 6: Smoke test

Cell 9: answerable and unanswerable questions

```
checks = [("Which optimizer was used for training?", "adam"),
          ("What hardware was the model trained on?", "p100"),
          ("Which data was used for English-German translation?", "wmt"),
          ("How is attention computed from queries, keys and values?", "dot")]
rows = []
for q, key in checks:
    r = ask(q)
    rows.append({"question": q, "kind": "answerable",
                 "has key phrase": key in r["answer"].lower(),
                 "valid citations": citations_ok(r["answer"], len(r["hits"])),
                 "refused": r["answer"] == REFUSAL})
for q in unanswerable:
    r = ask(q)
    rows.append({"question": q, "kind": "unanswerable",
                 "has key phrase": None, "valid citations": None,
                 "refused": r["answer"] == REFUSAL})
pd.DataFrame(rows)
```

YOUR TURN Complete the table below from your output. Then try a threshold of 0.4 and of 0.9 and describe what happens to answerable and to unanswerable questions.

Threshold	Answerable answered	Unanswerable refused	Comment
0.4			
your value			
0.9			

Part 7: A web interface

Cell 10: Gradio

```
import gradio as gr

history = []

def chat(question):
    standalone = condense(history, question)
    r = ask(standalone)
    history.append((question, r["answer"]))
    sources = "\n".join(f"[{i}] {h['source']}, page {h['page']}"
                        for i, h in enumerate(r["hits"], start=1))
    return r["answer"] + ("\n\nSources:\n" + sources if sources else "")

demo = gr.Interface(fn=chat, inputs=gr.Textbox(label="Your question"),
                    outputs=gr.Textbox(label="Answer"), title="Ask your files")
demo.launch(share=True)
```

The share link is temporary and public while the notebook runs; do not use private documents with it. Try five questions, including a follow-up and an unanswerable one.

Reflection questions

- Which failures came from retrieval and which from generation? How did you tell them apart?
- How would the app behave if two chunks contradicted each other? What could you add to the prompt or the pipeline?
- Your threshold came from seven questions. What could go wrong when you deploy it, and how would you check it?
- A larger hosted model would write better answers. Which improvements in this lab would still matter with it, and why?

Deliverable

Submit the executed notebook, the threshold with your reasoning, the completed table, a screenshot of the interface answering one answerable and one unanswerable question, and short answers to the reflection questions.

Troubleshooting

- **Out of memory when loading the model:** switch to Qwen/Qwen2.5-0.5B-Instruct, or enable a GPU runtime.
- **Generation is very slow:** on CPU this is expected; reduce k and max_new_tokens, or use a GPU runtime when one is available.
- **Answers ignore the context:** print the prompt and check that the right chunk is in it before changing anything else.